

Kick-starting Mini Projects

Module 1: Selecting a Suitable Project

How to Pick the Right Mini Project



Scope Feasibility

Keep the core feature set narrow (MVP). Avoid overly complex features that derail completion schedules.



Learning Value

Ensure the project introduces at least 1-2 new concepts or APIs you want to master rather than repeating known tech.



Real-World Utility

Build tools solving actual user pain points, automation tasks, or portfolio pieces that attract future employers.

Module 2: Project Planning & Requirements

Turning abstract ideas into structured roadmaps and specifications.

Defining Requirements & Milestones

Functional Requirements

- **User Actions:** What actions can users execute (e.g., register, filter data, export logs)?
- **Data Flow:** How data moves from user inputs through server endpoints to data stores.
- **Core Outputs:** UI dashboards, confirmation messages, or downloadable files.

Non-Functional & Milestones

- **Performance & UI:** Clean layout scaling across mobile and desktop.
- **Phase Breakdown:** Milestone 1 (Wireframe) to Milestone 3 (Deploy).
- **Risk Assessment:** Identifying bottlenecks (e.g., rate limits on external APIs).

Module 3: Technology Selection

Choosing the right frontend frameworks, backend engines, databases, and tools.

Selecting Your Stack

Layer	Beginner Friendly	Production / Scalable	Key Consideration
Frontend	HTML5, Vanilla CSS, JS	React, Next.js, Tailwind CSS	Component reusability vs setup speed
Backend	Node.js / Express, Python Flask	Spring Boot, FastAPI, Go	Asynchronous support & packages
Database	SQLite, LocalStorage, JSON	PostgreSQL, MongoDB, Redis	Data persistence needs & queries

Module 4: Development Approach

Adopting iterative execution, version control discipline, and clean code conventions.

Engineering Best Practices



Git Version Control: Commit early and often with descriptive messages. Use feature branches for major code additions.



MVP First Approach: Build the absolute core working loop before adding visual polish, animations, or extra features.



Incremental Testing: Test individual functions and API endpoints right after writing them rather than debugging everything at the end.



Early Deployment: Deploy your prototype to Vercel, Netlify, or Render on day one to catch environment config issues early.

Execution Schedule

1st Week

Ideation & Goal Setting: Brainstorming mini-project ideas and scoping down features.

Requirement & Tech Stack: Defining user flows and picking frontend/backend tools.

Next 3 Weeks

Architecture & Skeleton: Setting up boilerplate code and repository structure.

Execution & Review: Coding core MVP features, wrap-up, and Q&A discussion.

Summary & Action Plan

- ✓ Successful mini-projects rely on narrow, well-defined scopes and MVP execution.
- ✓ Disciplined requirement planning prevents feature creep during development cycles.
- ✓ Choose pragmatic tech stacks matching your skill level and target deliverables.
- ✓ Embrace incremental development and early deployment for continuous feedback.

Questions & Discussion